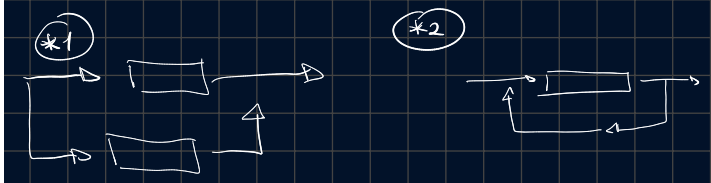
CPLP - Práctica 2

Ejercicio 1: Complete el siguiente cuadro:

Meta símbolos utilizados por		Símbolo utilizado en Diagramas sintacticos	Significado
BNF	EBNF		
palabra terminal	palabra terminal		Definición de un elemento terminal
<>	<>	rectángulo	Definición de un elemento no terminal
::=	::=	diagrama con rectángulos, óvalos y flechas	Meta símbolo, define una producción/regla
|	(|)	flecha que se divide en dos o más caminos	Selección de alternativa *1
< p > < p1 >	{ }	*2	Repetición
	*		Repetición de 0 o más veces
	+		Repetición de 1 o más veces
	[]		Opcional



Ejercicio 2: ¿Cuál es la importancia de la sintaxis para un lenguaje? ¿Cuáles son sus elementos?

Ejercicio 3: ¿Explique a qué se denomina regla lexicográfica y regla sintáctica?

Ejercicio 4:¿En la definición de un lenguaje, a qué se llama palabra reservadas? ¿A qué son equivalentes en la definición de una gramática? De un ejemplo de palabra reservada en el lenguaje que más conoce. (Ada,C,Ruby,Python,...)

2) La sintaxis es la forma de un lenguaje para definir cómo componer letras, dígitos y otros caracteres que permiten formar los programas. Permite que la comunicación entre el programador y la computadora sea clara. // Reglas para poder escribir código de un lenguaje.

Se compone de:

- Alfabeto/conjunto de caracteres: Símbolos permitidos en el lenguaje (cantidad finita).
- Identificadores: Cadena de caracteres [que determina el nombre de variables y funciones definidas por el programador.] DUDOSO // Cadena de caracteres que inicia por una letra. Depende del lenguaje los caracteres disponibles.
- Operadores: +, -, y otros que varían según el lenguaje.
- Comentarios y uso de blancos: Mejoran la legibilidad de un programa
- Palabras clave y reservadas → Palabras clave: Palabras con un significado en un contexto.
  - ↳ Palabras reservadas: Palabras clave que, además, no pueden ser usadas como identificador por el programador.

3) • Reglas lexicográficas: Conjunto de reglas para formar los "word", a partir de los caracteres del alfabeto.

• Reglas sintácticas: Conjunto de reglas que definen cómo formar expresiones y sentencias.

4) Las palabras clave son palabras con un significado especial dentro del lenguaje y que no pueden ser usadas para nombrar variables o funciones. Se corresponden con el conjunto de símbolos terminales de la gramática.

5) a) $G = (N, T, S, P)$ G = Gramática, reglas que definen un conjunto infinito de posibles sentencias válidas en el lenguaje.

N = Conjunto de símbolos NO terminales (<Numero_entero> y <Dígito>)
T = Conjunto de símbolos Terminales (0, 1, 2, ..., 9)
S = Símbolo distinguido de la gramática.
P = Conjunto de producciones de la gramática

Ejercicio 5: Dada la siguiente gramática escrita en BNF:

G= (N, T, S, P)

N = {<numero_entero>, <digito> }

T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}

S = <numero_entero>

P = {

<numero_entero>::=<digito><numero_entero> | <numero_entero><digito> | <digito>

<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

}

a- Identifique las componentes de la misma

b- Indique porqué es ambigua y corríjala

b) Es ambiguo ya que uso recursión por izquierda y por derecha, esto puede generar problemas en el árbol de sintáxis.

P = { <Numero_entero> ::= <dígito> | <dígito> <Numero_entero>
<Digito> ::= 0|1|2|3|...|9.
}

Ejercicio 6: Defina en BNF (Gramática de contexto libre desarrollada por Backus- Naur) la gramática para la definición de una palabra cualquiera.

G = (N, T, S, P)

N = {<Palabra>, <Letra>}
T = { a, b, c, d, ... z, A, B, C, ..., Z}
S = <Palabra>
P = {
<Palabra> ::= <Letra> | <Letra> <Palabra>
<Letra> ::= a | b | c | d | ...
}

N = {<Palabra>, <Mayus>, <Minus>}
T = {A, B, C, ... Z, a, b, c, ..., z}
S = <Palabra>
P = {
<Palabra> ::= <Mayus> | <Minus> | <Mayus> <Palabra> | <Minus> <Palabra>
<Mayus> ::= A | B | C | ... | Z
<Minus> ::= a | b | c | d | ... | z
}

Ejercicio 7: Defina en EBNF la gramática para la definición de números reales. Inténtelo desarrollar para BNF y explique las diferencias con la utilización de la gramática EBNF.

7) G = (N, T, S, P) EBNF

N = {<Numero>, <dígito>}
T = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, "+", "-", "."}
S = <Numero>
P = {
<Numero> ::= [-] {<Digito>} [+ {<Digito>}]*
<Digito> ::= 0|1|2|3|4|5|6|7|8|9
}

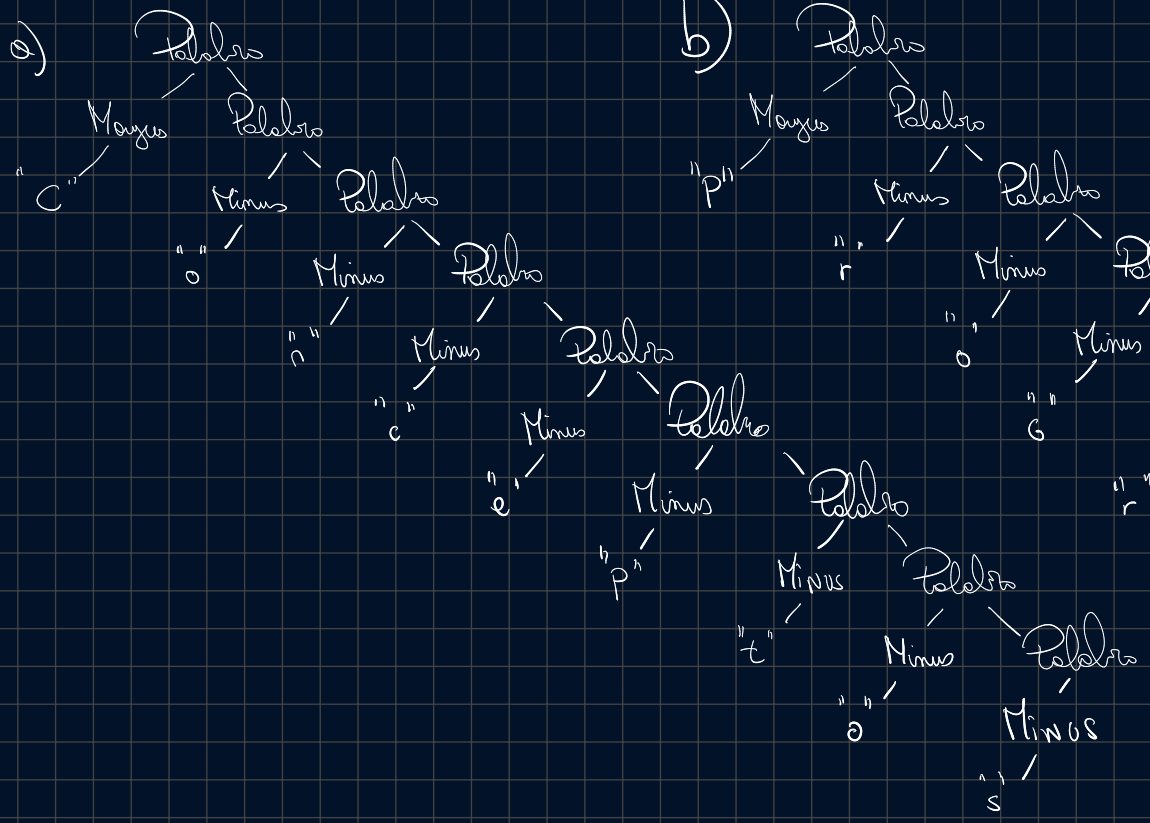
BNF N = {<Numero>, <entero>, <dígito>}
P = {
<Numero> ::= -<entero> | +<entero> |
- <entero>, <entero> | + <entero>, <entero>
<entero> ::= <dígito> | <dígito> <entero>
<dígito> ::= 0|1|2|3|4|5|6|7|8|9
}
(No contemplo los números sin signo como algo != de +)

Ejercicio 8: Utilizando la gramática que desarrolló en los puntos 6 y 7, escriba el árbol sintáctico de:

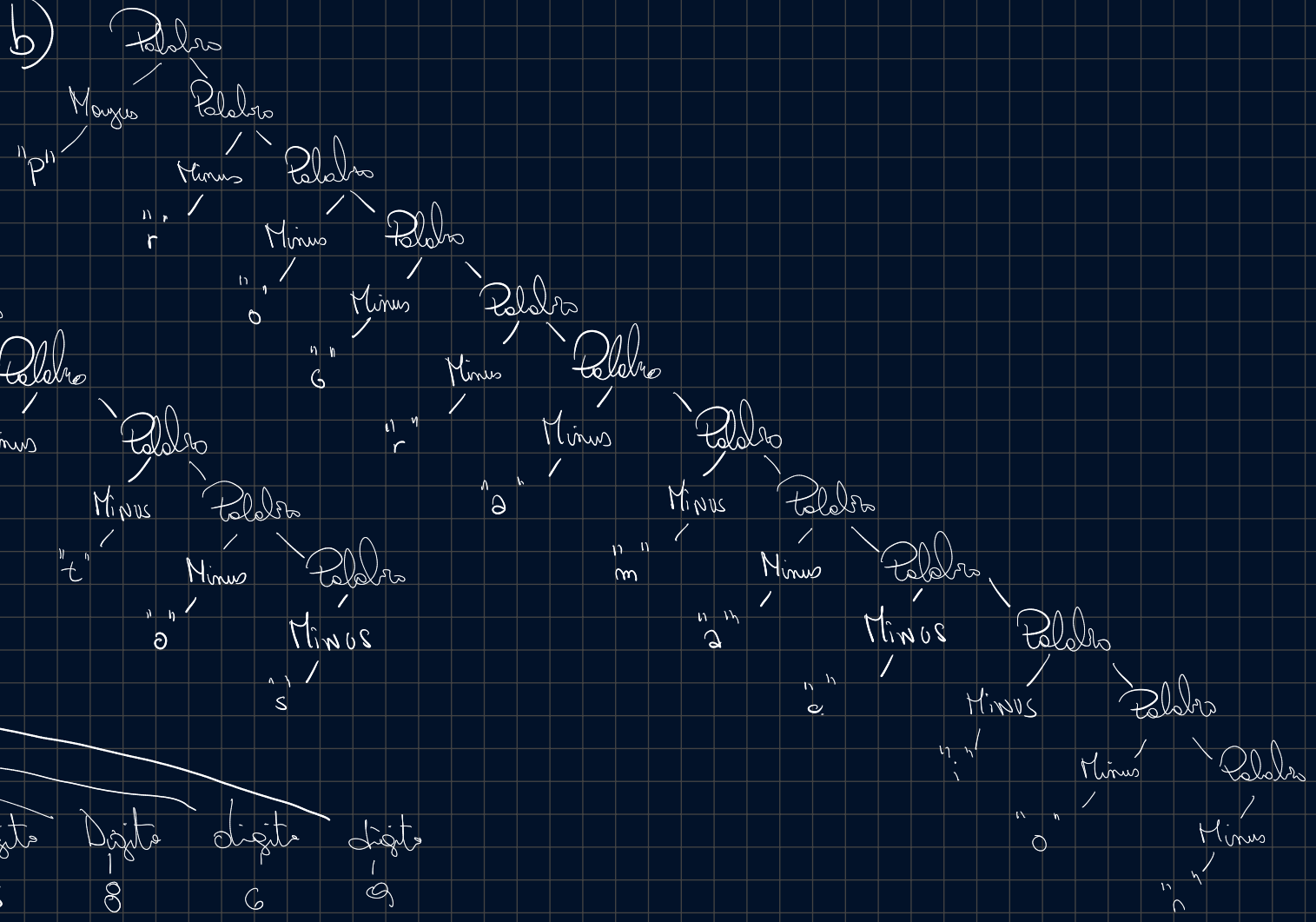
- **a.** Conceptos
- **b.** Programación
- **c.** 1255869
- **d.** 854,26
- **e.** Conceptos de lenguajes

8)

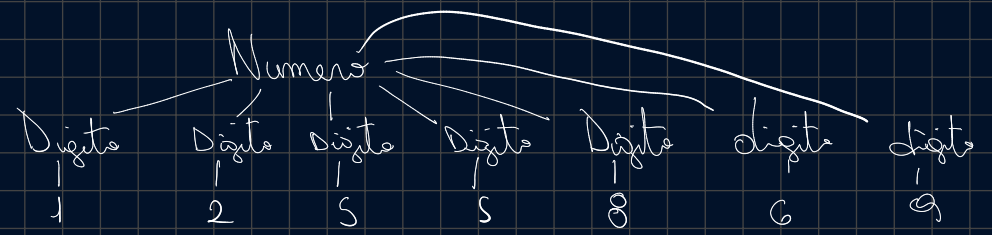
a)



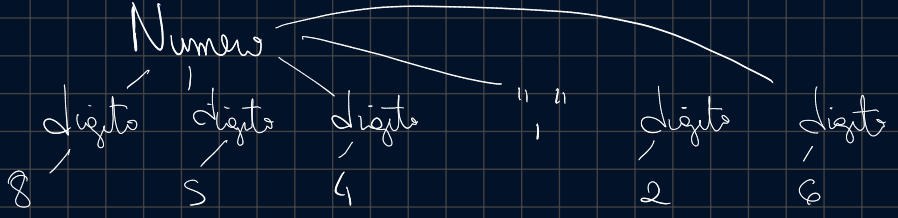
b)



c)

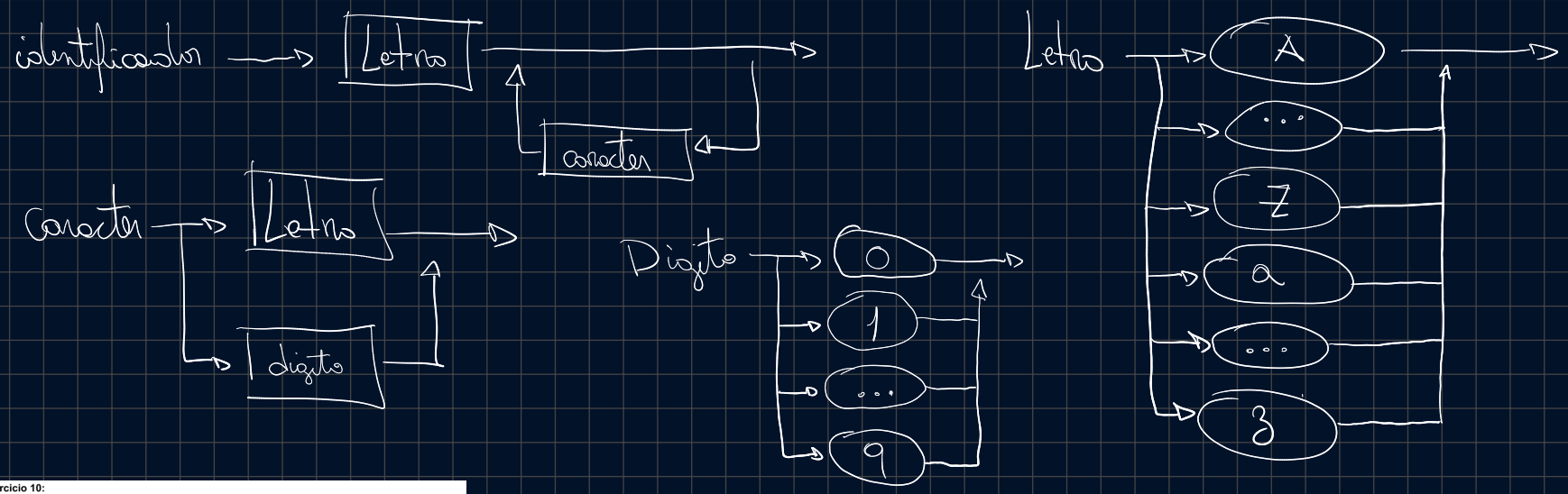


d)



e) Con la gramática definida solo pueden expresarse palabras, debería definir una gramática para frases que contengan espacios.

Ejercicio 9: Defina utilizando diagramas sintácticos la gramática para la definición de un identificador de un lenguaje de programación. Tenga presente como regla que un identificador no puede comenzar con números.



Ejercicio 10:

a) Defina con EBNF la gramática para una expresión numérica, dónde intervienen variables y números. Considerar los operadores +, -, * y / sin orden de prioridad. No considerar el uso de paréntesis.

b) A la gramática definida en el ejercicio anterior agregarle prioridad de operadores.

c) Describa con sus palabras los pasos y decisiones que tomó para agregarle prioridad de operadores al ejercicio anterior.

a) $G = (N, T, S, P)$

$N = \{\langle expresion \rangle, \langle Variable \rangle, \langle Numero \rangle\}$

$T = \{a, \ldots, z, A, \ldots, Z, 0, \ldots, 9, +, -, *, /\}$

$S = \langle expresion \rangle$

$P = \{$

$\langle expresion \rangle ::= (\langle Variable \rangle | \langle Numero \rangle) \{(+|-|*|/)(\langle Variable \rangle | \langle Numero \rangle)\}+$

$\langle Variable \rangle ::= \langle Letra \rangle \{(\langle Letra \rangle | \langle digito \rangle)\}*$

$\langle Numero \rangle ::= [-] \{\langle digito \rangle\}+$

$\langle digito \rangle ::= (0|1|2|\ldots|9)$

$\langle Letra \rangle ::= (a|b|\ldots|z|A|B|\ldots|Z)$

$\}$

b) P = { <expresion> ::= <termino> {(+|-) <termino>}*

<termino> ::= (<Variable>|<Numero>) {(*|/) (<Variable>|<Numero>)}*

<Variable> ::= <Letra> {(<Letra>|<dígito>)}*

<Numero> ::= [-] {<digito>}+

<digito> ::= (0|1|2|...|9)

<Letra> ::= (a|b|...|z|A|B|...|Z)

G = (N, T, S, P)

N = { <expresion>, <termino>, <variable>, <Numero>, <digito> }

T = {a ... z, A ... Z, 0 ... 9}

S = <expresion>

c) Lo que hice para definir esa prioridad fue que los sumas y restas se apliquen sobre término, donde término es un no terminal sobre el cual se hacen * y /, esto resulta en que se resuelvan primero esas operaciones, y, luego, las + y -.

Ejercicio 11: La siguiente gramática intenta describir sintácticamente la sentencia for de ADA, indique cuál/cuáles son los errores justificando la respuesta.

```
N= {<sentencia_for>,  <bloque>, <variable>, <letra>, <cadena>, <digito>, <otro>, <operacion>,
 <llamada_a_funcion>, <numero>,  <sentencia> }
 P= { <sentencia_for>::= for (i= IN 1..10) loop <bloque> end loop;
<variable>::= <letra> | <cadena>
<cadena>::= { ( <letra> | <digito> | <otro> ) }+
<letra>::=( a | .. | z | A | .. | Z )
<digito>::= ( 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 0 )
<bloque>::=  <sentencia> | <sentencia> <bloque> | <bloque> <sentencia> ;
<sentencia>::= <sentencia_asignacion> | <llamada_a_funcion> | <sentencia_if> |
<sentencia_for> |  <sentencia_while> | <sentencia_switch> }
```

1) No define la gramática (G = (N, T, S, P))

2) No define T (Terminales) ni S (Símbolo distinguido)

3) P no define <operación>, <otro>, <llamada_a_funcion>, <Numero>

4) N no define <sentencia_asignación>, <sentencia_if>, <sentencia_while>, <sentencia_switch>

5) En la definición de <Bloque> se determina "<Bloque> <sentencia> | <sentencia> <Bloque>", esto genera una producción ambigua ya que puede generar múltiples árboles posibles, haciéndola ambigua.

Ejercicio 12: Realice en EBNF la gramática para la definición un tag div en html 5. (Puede ayudarse con el siguiente enlace (https://developer.mozilla.org/es/docs/Web/HTML/Elemento/div)

P = { <Div> ::= "<Div>" {(<HTML>|<PALABRA>)}* "</Div>"

<HTML> ::= <Tag> {(<HTML>|<PALABRA>)}* </Tag>

<Tag> ::= "<" <Palabra> ">"

</Tag> ::= "</" <Palabra> ">"

<Palabra> ::= {(<Letra>|<digito>)}+

<Letra> ::= (a|b|c|...|z|A|B|C|...|Z)

<digito> ::= (0|1|...|9)

G = (N, T, S, P)

N = {<Div>, <HTML>, <Palabra>, <Tag>, </Tag>, <Letra>, <digito>}

S = <Div>

T = {a, b, ..., z, A, B, ..., Z, 0, ..., 9}

|| En caso de no poder usar "/" en un no-terminal, pongo <TagCierre>

$P = \{$

$\langle Func \rangle ::= \text{"function"} \langle Palabra \rangle \text{"("} \{ \langle Variable \rangle \}^* \text{")"} \{ \text{"}\langle codigo \rangle\text{"} \}$

$\langle Variable \rangle ::= \text{"\$"} (\langle Palabra \rangle \mid \langle Palabra \rangle \text{","} \langle Variable \rangle)$

$\langle Palabra \rangle ::= \langle char \rangle \{ (\langle char \rangle \mid \langle digito \rangle) \}^*$

$\langle char \rangle ::= (\text{"_"} \mid a \mid \ldots \mid z \mid A \mid \ldots \mid Z)$

$\langle digito \rangle ::= (0 \mid 1 \mid \ldots \mid 9)$

$\}$

$G = (N, T, S, P)$

$N = \{ \langle Func \rangle, \langle Palabra \rangle, \langle Variable \rangle, \langle char \rangle, \langle digito \rangle, \langle codigo \rangle \}$

$T = \{ a, \ldots, z, A, \ldots, Z, 0, \ldots, 9 \}$

en PHP:

```
function <Nombre> (<Parametros>)
{
    <Codigo>
}
```

1) En PHP las variables van con $

↳ $contador = 0;

2) Los parametros pueden no llevar tipo

Ejercicio 14: Sobre un lenguaje de su preferencia escriba en EBNF la gramática para la definición de funciones o métodos o procedimientos (considere los parámetros en caso de ser necesario)